

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №13

по теме:
РЕАЛИЗАЦИЯ ОДНОВРЕМЕННОЙ ПЕРЕДАЧИ И ПРИЕМА НА ДВУХ
SDR. РЕАЛИЗАЦИЯ ФУНКЦИЙ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахпашиев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ПРАКТИКА	4
2.1	Работа с двум SDR.....	4
2.2	Идея	4
2.3	Реализация.....	4
2.4	Тестирование схемы Гарднера	9
3	ВЫВОД	10

ВВЕДЕНИЕ

1.1 Введение

На прошлом занятии мы полностью реализовали схему Гарднера. На этом занятии протестируем ее работу.

Также до этого момента мы передавали сигнал на одной SDR (включали передачу и прием одновременно). Как оказалось, это не самая хорошая идея, поэтому на этом занятии организуем работу с SDR так, чтобы можно было работать с двумя SDR (одна на TX, другая на RX).

1.2 Цель

Протестировать работу схемы Гарднера и добавить в проект функционал, позволяющий удобно работать с двумя SDR.

ПРАКТИКА

2.1 Работа с двум SDR

2.2 Идея

Добавим возможность при запуске бинарника передавать через консоль параметры для SDR: uri, mode, time, где uri - адрес SDR (таким образом мы сможем удобно выбрать конкретную SDR из двух, подключенных к хосту), mode - режим работы SDR (0 - RX, 1 - TX), time - время, которое SDR будет работать (секунды!). В программе будем анализировать аргументы и запускать SDR нужным образом.

Дополнительно для удобства добавим структуру, которая будет хранить настройки SDR. Прием и передачу разобьем на отдельные функции

2.3 Реализация

Из функций реализуем отдельную библиотеку. Заголовочный файл:

```
#include <SoapySDR/Device.h>
#include <SoapySDR/Formats.h>
#include <complex.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

/**
 * @brief Config for init Pluto SDR
 */
struct sdr_config_t
{
    char *usb_uri;    /**< uri address of pluto (use iio_info -a to
                      see connected plutos)*/
    int buffer_size; /**< count of I/Q samples, that SDR
                      receive/send for one time */
    int tx_sample_rate;
    int tx_carrier_freq;
```

```

float tx_gain;
int rx_sample_rate;
int rx_carrier_freq;
float rx_gain;
int channels[1] = {0};
};

/**
 * @brief create sdr object
 *
 * @param config pluto config
 * @return pluto object
 */
struct SoapySDRDevice *setup_pluto_sdr(sdr_config_t *config);

/**
 * @brief setup streams
 *
 * @param sdr sdr object
 * @param config sdr config
 * @param isRx 0-setup TX stream, 1-setup RX stream
 * @return stream object
 */
struct SoapySDRStream *setup_stream(struct SoapySDRDevice *sdr,
                                     sdr_config_t *config, bool
                                     isRx);

/**
 * @brief close streams, free memory. Finished work with pluto
 *
 * @param sdr sdr object
 * @param rxStream
 * @param txStream
 *
 * @return stream object
 */
void close_pluto_sdr(struct SoapySDRDevice *sdr, SoapySDRStream
                    *rxStream,
                    SoapySDRStream *txStream);

/**
 * @brief start receiving samples and write their in file

```

```

*
* @param sdr sdr object
* @param rx_stream rx stream object
* @param rx_buffer how many samples sdr receive for one time
* @param buff_size how many samples sdr receive for one time
* @param rx_mtu Max Transmission Unit
* @param filename file, where will be writing receive samples
* @param work_time time, that sdr will receive signal
*/
void start_rx(struct SoapySDRDevice *sdr, SoapySDRStream
    *rxStream,
                int16_t *rx_buffer, int buff_size, int rx_mtu,
                char *filename,
                int work_time);

/**
* @brief start transmit samples
*
* @param sdr_object sdr object
* @param tx_stream
* @param rx_stream
* @param tx_samples samples that will transmitted
* @param tx_samples_count size of tx samples
* @param rx_buffer
* @param tx_buffer
* @param filename file, where will be writing tx samples
* @param work_time time, that sdr will tx signal
*/
void start_tx(struct SoapySDRDevice *sdr, SoapySDRStream
    *txStream,
                SoapySDRStream *rxStream, int16_t *tx_samples,
                int tx_samples_count, int16_t *rx_buffer, int16_t
                *tx_buffer,
                int buff_size, int work_time);

```

Реализацию функций можно посмотреть в srcmainpluto_lib

Пример работы с SDR:

```

/*
    This code work with Pluto SDR. He init and setup SDR,
    receive/transmit samples

```

```

*/

#include <SoapySDR/Device.h>
#include <SoapySDR/Formats.h>
#include <complex.h>
#include <iostream>
#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>
#include <vector>

#include "../includes/general/subfuncs.hpp"
#include "../includes/pluto_lib/pluto_lib.hpp"

int main(int argc, char *argv[]) {

    /*check arguments*/
    if (argc != 4) {
        printf("Programm wait 3 argument:\n \
            1.char* pluto usb_uri (example: )\n \
            2.int rx or tx (0-rx, 1-tx)\n \
            3.int time to work (in seconds)");

        return 1;
    }

    /*tx or rx mode*/
    int mode = std::stoi(argv[2]);

    int time_to_work = std::stoi(argv[3]);

    /*create config for SDR*/
    sdr_config_t config;
    config.usb_uri = argv[1];
    config.buff_size = 1920;
    config.rx_carrier_freq = 800e6;
    config.tx_carrier_freq = 800e6;
    config.rx_sample_rate = 1e6;
    config.tx_sample_rate = 1e6;
    config.rx_gain = 35.0;
    config.tx_gain = -15.0;

```

```

/*files for tx/rx samples*/
char *rxdata = "../pcm/rxdata.pcm";
char *txdata = "../pcm/txdata.pcm";

/*setup pluto*/
struct SoapySDRDevice *sdr = setup_pluto_sdr(&config);
struct SoapySDRStream *rxStream = setup_stream(sdr, &config,
    1);
struct SoapySDRStream *txStream = setup_stream(sdr, &config,
    0);

/*create buffers*/
int16_t tx_buffer[2 * config.buff_size];
int16_t rx_buffer[2 * config.buff_size];

/*if we want only rx*/
if (mode == 0) {
    start_rx(sdr, rxStream, rx_buffer, config.buff_size,
        config.buff_size,
        rxdata, time_to_work);
/*if we want only tx*/
} else if (mode == 1) {
    /*read sample as std::complex*/
    std::vector<std::complex<int16_t>> complex_samples =
        read_pcm(std::string("../pcm/tx_samples.pcm"));
    /*create array in format: I0,Q0,I1,Q1 ... In,Qn*/
    int uncomplex_samples_size = complex_samples.size() * 2;
    std::vector<int16_t> uncomplex_samples;
    uncomplex_samples.reserve(uncomplex_samples_size);

    for (int i = 0; i < complex_samples.size(); ++i) {
        uncomplex_samples.push_back(complex_samples[i].real());
        uncomplex_samples.push_back(complex_samples[i].imag());
    }

    /*start tx*/
    start_tx(sdr, txStream, rxStream, rx_buffer,
        uncomplex_samples.data(),
        complex_samples.size(), config.buff_size, txdata,
        time_to_work);
} else {
    printf("Invalid mode. Enter mode=0 or mode=1");
}

```



```

}

/*free resources*/
close_pluto_sdr(sdr, rxStream, txStream);

return EXIT_SUCCESS;
}

```

2.4 Тестирование схемы Гарднера

Для проверки схемы Гарднера я отправлю некоторый текст, а потом сделаю прием без схемы Гарднера и со схемой Гарднера

Отправляемый текст:

```

21 std::string message(
22     "Mr. and Mrs. Smith have one son and one daughter. The son's name is "
23     "John. The daughter's name is Sarah. The Smiths live in a house. They "
24     "have a living room. They watch TV in the living room. The father cooks "
25     "food in the kitchen. They eat in the dining room. The house has two "
26     "bedrooms. They sleep in the bedrooms. They keep their clothes in the "
27     "closet. There is one bathroom. They brush their teeth in the bathroom. "
28     "The house has a garden. John and Sarah play in the garden. They have a "
29     "dog. John and Sarah like to play with the dog."); // tx message
30

```

Рисунок 1 — Отправляемый текст

Без Гарднера:

```

[file_logger] [info] message: u8, an u8p, eph i uU DELU DELU EOP o eT uhtur. TiU DELDEL DEL ep J hn. THU PEUW LUB'S  B. THU S

```

Рисунок 2 — Принятое сообщение (без Гарднера)

Видим полную ”кашу” из символов. Ничего общего это сообщение не имеет с отправляемым

С Гарднером

```

[file_logger] [info] message: Mr. and Mzs. Smi h ha e one s a dnu e dauhter. U e DELDELh's naee is Join. The Tauwht r name s Sara

```

Рисунок 3 — Принятое сообщение (с Гарднером)

Сообщение не идеальное, но уже очень похоже на отправляемое. Результат очень зависит от параметров самой схемы Гарднера и нужно правильно ее настроить.

ВЫВОД

В ходе работы я протестировал схему Гарднера и реализовал удобную работу с двумя SDR с одного хоста.